Titanic.ipynb

File Edit View Insert Runtime Tools Help

Share

Commands + Code + Text Run all

RAM Disk

[80] ✓ 0s

```
# install all libraries
```

[81] ✓ 0s

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

[82] ✓ 0s

```
# load the dataset -
# create df variable

# df = pd.read_csc("titanic.csv)
```

[83] ✓ 0s

```
df = sns.load_dataset("titanic")
```

[84] ✓ 0s

```
# display the first 5 rows
```

[85] ✓ 0s

```
df.head()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who
0	0	3	male	22.0	1	0	7.2500	S	Third	mar
1	1	1	female	38.0	1	0	71.2833	C	First	womar
2	1	3	female	26.0	0	0	7.9250	S	Third	womar
3	1	1	female	35.0	1	0	53.1000	S	First	womar
4	0	3	male	35.0	0	0	8.0500	S	Third	mar

Next steps: Generate code with df | New interactive sheet

[86] ✓ 0s

```
# see all columns name
```

[87] ✓ 0s

```
df.columns
```

```
Index(['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare',
       'embarked', 'class', 'who', 'adult_male', 'deck', 'embark_town',
       'alive', 'alone'],
      dtype='object')
```

[88] ✓ 0s

```
# Undestand the dataset
```

[89] ✓ 0s

```
# rows and columns
```

[90] ✓ 0s

```
df.shape
```

```
(891, 15)
```

[91] ✓ 0s

```
# statistical summery
```

[92] ✓ 0s

```
df.describe()
```

	survived	pclass	age	sibsp	parch	fare
count	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

[93] ✓ 0s

```
# we have 891 passengers with 15 columns / featurs
```

[94] ✓ 0s

```
# check missing values
```

[95] ✓ 0s

```
df.isnull().sum()
```

	0
survived	0
pclass	0
sex	0
age	177
sibsp	0
parch	0
fare	0

embarked	2
class	0
who	0
adult_male	0
deck	688
embark_town	2
alive	0
alone	0

dtype: int64

[96] ✓ 0s

```
# missing values in percenatage
```

[97] ✓ 0s

```
df.isnull().mean()*100
```

	0
survived	0.000000
pclass	0.000000
sex	0.000000
age	19.865320
sibsp	0.000000
parch	0.000000
fare	0.000000
embarked	0.224467
class	0.000000
who	0.000000
adult_male	0.000000
deck	77.216611
embark_town	0.224467
alive	0.000000
alone	0.000000

dtype: float64

[98] ✓ 0s

```
missing = df.isnull().sum()
```

[99] ✓ 0s

```
missing[missing > 0]
```

	0
age	177
embarked	2
deck	688
embark_town	2

dtype: int64

[100] ✓ 0s

```
# create a copy before cleaning
```

[101] ✓ 0s

```
# new variable name raw_df and clean_df
```

[102] ✓ 0s

```
raw_df = df
```

[103] ✓ 0s

```
clean_df = df
```

[104] ✓ 0s

```
# same data in df , raw_df, clean_df
```

[105] ✓ 0s

```
# Check for the duplicate records
```

[106] ✓ 0s

```
clean_df.duplicated().sum()
```

```
np.int64(107)
```

[107] ✓ 0s

```
# remove 107 duplicate records
```

[108] ✓ 0s

```
clean_df = clean_df.drop_duplicates()
```

[109] ✓ 0s

```
clean_df.shape
```

```
(784, 15)
```

[110] ✓ 0s

```
# we have unique 784 customers
```

[111] ✓ 0s

```
# missing values - remove - replace ?
```

[112] ✓ 0s

```
clean_df.isnull().sum()
```

	0

survived	0
pclass	0
sex	0
age	106
sibsp	0
parch	0
fare	0
embarked	2
class	0
who	0
adult_male	0
deck	582
embark_town	2
alive	0
alone	0

dtype: int64

[113] ✓ 0s

```
# age is numerical data having 106 missing values
# replace by ? - average (median)

# mean - mean robhust to outlier
# we use the median
```

[114] ✓ 0s

```
clean_df["age"].median()
```

```
28.25
```

[115] ✓ 0s

```
#clean_df["age"] = clean_df["age"].fillna(28.25)

clean_df["age"] = clean_df["age"].fillna(clean_df["age"].median())

#subqueery
```

```
/tmp/ipykernel_1455/2875217557.py:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/st
  clean_df["age"] = clean_df["age"].fillna(clean_df["age"].median())
```

[116] ✓ 0s

```
clean_df["age"].isnull().sum()
```

```
np.int64(0)
```

[117] ✓ 0s

```
# embarked is the categorical feature
```

[118] ✓ 0s

```
# so replace with - mode
```

[119] ✓ 0s

```
clean_df["embarked"].mode()
```

	embarked
0	S

dtype: object

[120] ✓ 0s

```
clean_df["embarked"].mode()[0]
# [0]  - for indexing
```

```
'S'
```

[121] ✓ 0s

```
clean_df["embarked"] =  clean_df["embarked"].fillna(clean_df["embarked"].mode
```

```
/tmp/ipykernel_1455/2864406080.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/st
  clean_df["embarked"] =  clean_df["embarked"].fillna(clean_df["embarked"].mod
```

[122] ✓ 0s

```
clean_df["embarked"].isnull().sum()
```

```
np.int64(0)
```

[123] ✓ 0s

```
# same for embarked town
```

[124] ✓ 0s

```
clean_df["embark_town"] =  clean_df["embark_town"].fillna(clean_df["embark_to
```

```
/tmp/ipykernel_1455/1537302128.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/st
  clean_df["embark_town"] =  clean_df["embark_town"].fillna(clean_df["embark_t
```

[125] ✓ 0s

```
clean_df["embark_town"].isnull().sum()
```

```
np.int64(0)
```

[126] ✓ 0s

```
# we drop the dec column
```

[127] ✓ 0s

```
clean_df = clean_df.drop(columns=["deck"])
```

[128] ✓ 0s

```
clean_df.isnull().sum()
```

	0
survived	0
pclass	0
sex	0
age	0
sibsp	0
parch	0
fare	0
embarked	0
class	0
who	0
adult_male	0
embark_town	0
alive	0
alone	0

dtype: int64

[129] ✓ 0s

```
# Data Types ?
```

[130] ✓ 0s

```
clean_df.dtypes
```

	0
survived	int64
pclass	int64
sex	object
age	float64
sibsp	int64
parch	int64

fare	float64
embarked	object
class	category
who	object
adult_male	bool
embark_town	object
alive	object
alone	bool

dtype: object

[131] ✓ 0s
```
# conver object data type into category
```

[132] ✓ 0s
```
clean_df["sex"] = clean_df["sex"].astype("category")
```

[133] ✓ 0s
```
clean_df["embarked"] = clean_df["embarked"].astype("category")
```

[134] ✓ 0s
```
clean_df["who"] = clean_df["who"].astype("category")
```

[135] ✓ 0s
```
clean_df["embark_town"] = clean_df["embark_town"].astype("category")
```

[136] ✓ 0s
```
clean_df["alive"] = clean_df["alive"].astype("category")
```

[137] ✓ 0s
```
clean_df.dtypes
```

	0
survived	int64
pclass	int64
sex	category
age	float64
sibsp	int64
parch	int64
fare	float64
embarked	category
class	category
who	category

adult_male	bool
embark_town	category
alive	category
alone	bool

dtype: object

[138] ✓ 0s

```
# check for the outliers
```

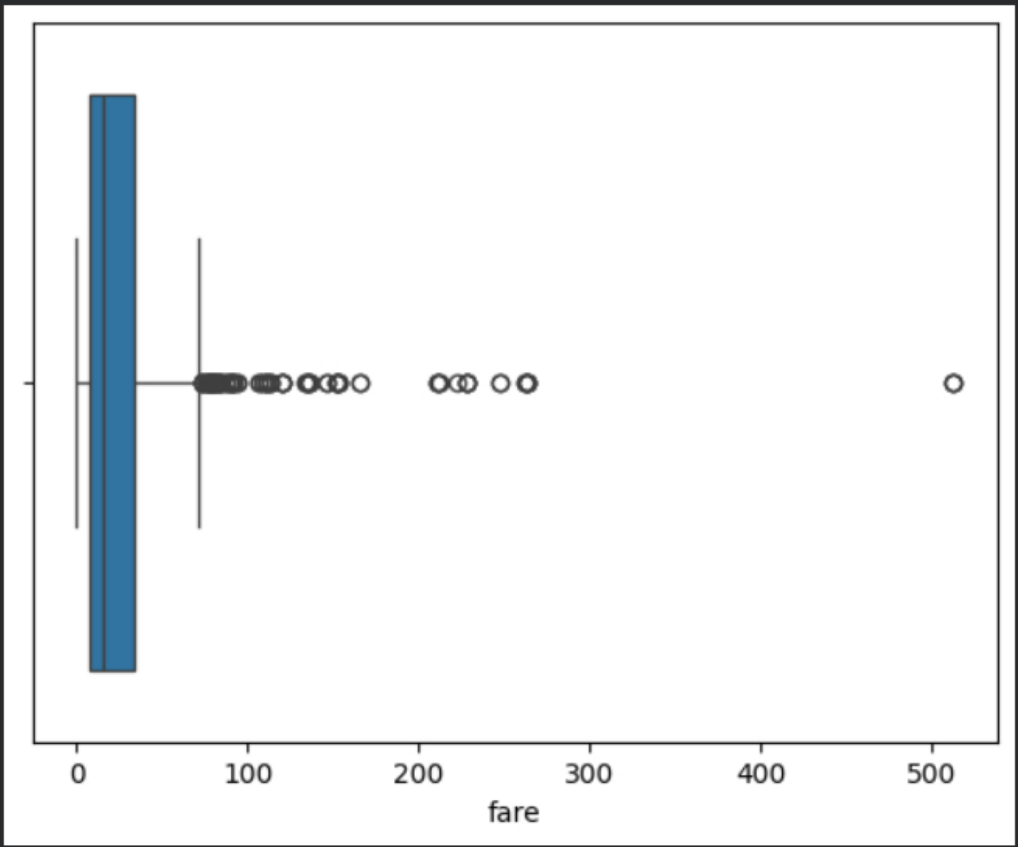
[139] ✓ 0s

```
# 5 number summery - box plot -
# Q1, Q2, Q3, IQR, LB, UB
```

[140] ✓ 0s

```
sns.boxplot(data= clean_df, x= "fare")
```

```
<Axes: xlabel='fare'>
```



[141] ✓ 0s

```
Q1 = clean_df["fare"].quantile(0.25)
```

[142] ✓ 0s

```
Q2 = clean_df["fare"].quantile(0.50)
```

[143] ✓ 0s

```
Q3 = clean_df["fare"].quantile(0.75)
```

[144] ✓ 0s

```
IQR = Q3 - Q1
```

[145] ✓ 0s
```
lower_limit = Q1 - 1.5 * IQR
```

[146] ✓ 0s
```
upper_limit = Q3 + 1.5 * IQR
```

[147] ✓ 0s
```
outliers = clean_df[
    (clean_df["fare"] < lower_limit) |
    (clean_df["fare"] > upper_limit)
]
```

[148] ✓ 0s
```
len(outliers)
```
```
102
```

[149] ✓ 0s
```
# data cleaning
clean_df["sex"].unique()
```
```
['male', 'female']
Categories (2, object): ['female', 'male']
```

[150] ✓ 0s
```
clean_df["sex"] = clean_df["sex"].astype("string").str.upper()
```

[151] ✓ 0s
```
clean_df["sex"].unique()
```
```
<StringArray>
['MALE', 'FEMALE']
Length: 2, dtype: string
```

[152] ✓ 0s
```
# feature engg
```

[153] ✓ 0s
```
clean_df["age_group"] = pd.cut(
    clean_df["age"],
    bins = [0,12,18,35,60,100],
    labels =["child", "teen", "adult", "senior", "oldage"]
)
```

[154] ✓ 0s
```
clean_df["age_group"].unique()
```
```
['adult', 'senior', 'child', 'teen', 'oldage']
Categories (5, object): ['child' < 'teen' < 'adult' < 'senior' < 'oldage']
```

[155] ✓ 0s
```
clean_df.head()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	w
0	0	3	MALE	22.0	1	0	7.2500	S	Third	m
1	1	1	FEMALE	38.0	1	0	71.2833	C	First	wom

2	1	3	FEMALE	26.0	0	0	7.9250	S	Third	wom
3	1	1	FEMALE	35.0	1	0	53.1000	S	First	wom
4	0	3	MALE	35.0	0	0	8.0500	S	Third	m

Next steps: Generate code with clean_df | New interactive sheet

[156] ✓ 0s
```
clean_df["family_size"]= clean_df["sibsp"] + clean_df["parch"]+1
```

[157] ✓ 0s
```
# EDA - exploratory data analysis
```

[158] ✓ 0s
```
clean_df.columns
```
```
Index(['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare',
       'embarked', 'class', 'who', 'adult_male', 'embark_town', 'alive',
       'alone', 'age_group', 'family_size'],
      dtype='object')
```

[159] ✓ 0s
```
#qu1 - total passenger
```

[161] ✓ 0s
```
clean_df.shape
```
```
(784, 16)
```

[162] ✓ 0s
```
# 784 total passengers
```

[164] ✓ 0s
```
#qu2 - what is the overall survival rate
```

[165] ✓ 0s
```
clean_df["survived"].value_counts()
```

survived	count
0	461
1	323

dtype: int64

[166] ✓ 0s
```
clean_df["survived"].value_counts(normalize=True)*100
```

survived	proportion
0	58.80102

1 41.19898

dtype: float64

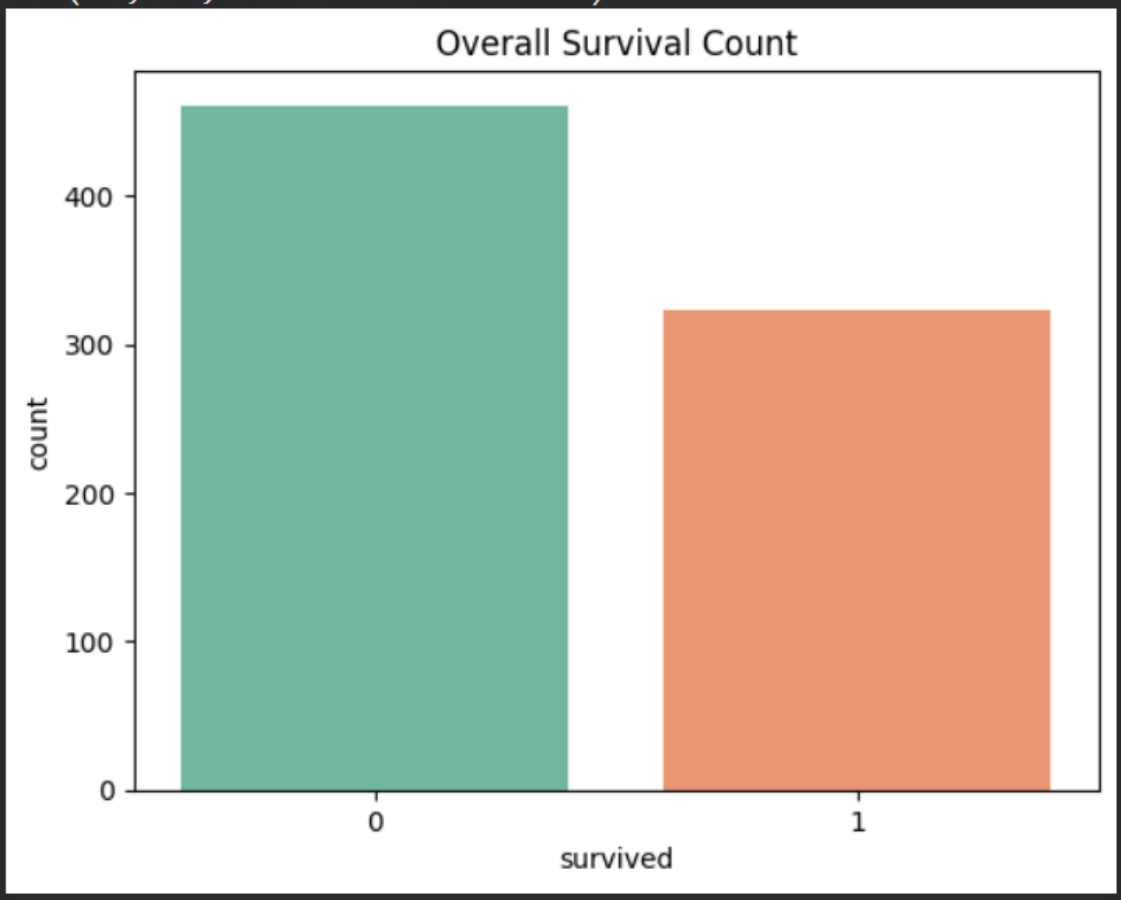
[169] ✓ 0s

```
sns.countplot(data= clean_df , x="survived", palette="Set2")
plt.title("Overall Survival Count")
```

```
/tmp/ipykernel_1455/4215191691.py:1: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in

  sns.countplot(data= clean_df , x="survived", palette="Set2")
Text(0.5, 1.0, 'Overall Survival Count')
```



[171] ✓ 0s

```
# qu3 - What was the survival rate by gender?
# survial & gender & count
# cat vs cat  - crosstab
```

[172] ✓ 0s

```
pd.crosstab(clean_df["sex"] , clean_df["survived"],
            normalize=True) *100
```

survived	0	1
sex		
FEMALE	9.693878	27.678571
MALE	49.107143	13.520408

[173] ✓ 0s

```
# pivot table
```

[174] ✓ 0s

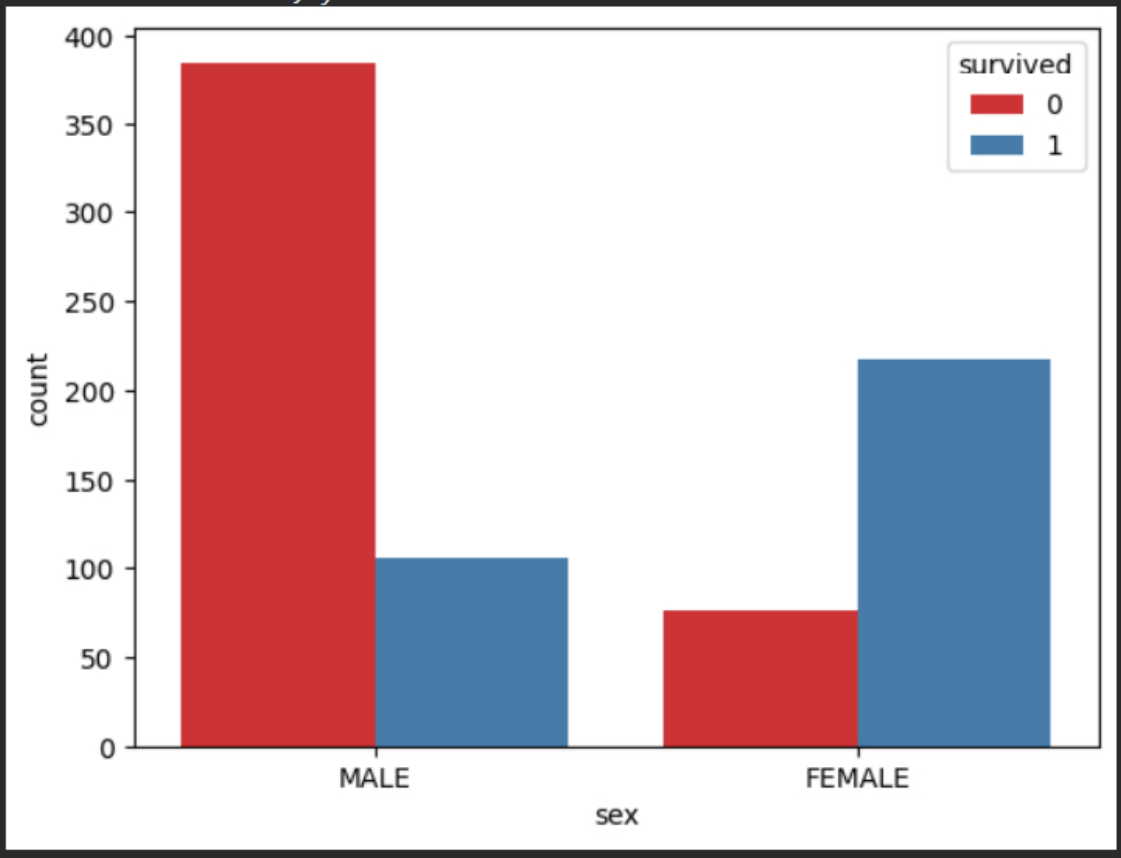
```
df.pivot_table(index ="sex", columns="alive",
               values="survived", aggfunc="count",
               fill_value=0)
```

alive	no	yes
sex		
female	81	233
male	468	109

[177] ✓ 0s

```
sns.countplot(data= clean_df,  x="sex", hue="survived",
              palette=("Set1"))
```

```
<Axes: xlabel='sex', ylabel='count'>
```



[179] ✓ 0s

```
clean_df.groupby("sex")["survived"].value_counts()
```

		count
sex	survived	
FEMALE	1	217
	0	76

MALE	0	385
	1	106

dtype: int64

[180] ✓ 0s

```
clean_df.groupby("sex")["survived"].value_counts().unstack()
```

survived	0	1
sex		
FEMALE	76	217
MALE	385	106

[181] ✓ 0s

```
#qu4 - how did passenger survival by pclass
```

[182] ✓ 0s

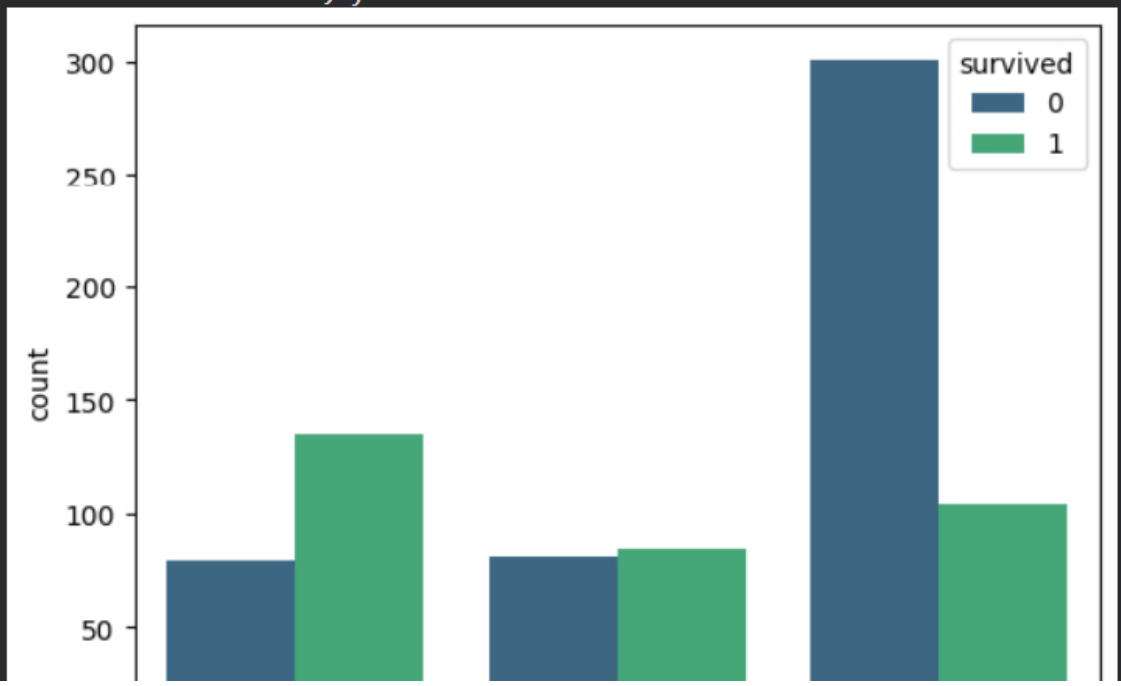
```
pd.crosstab(clean_df["class"] , clean_df["survived"])
```

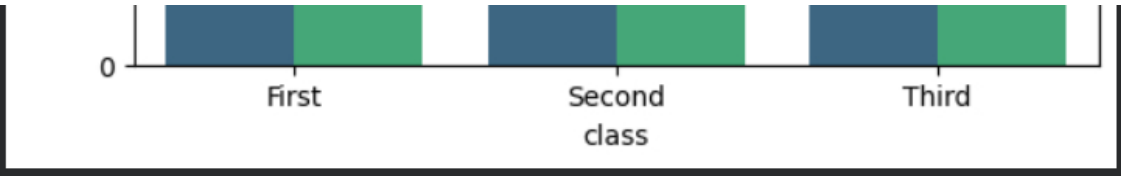
survived	0	1
class		
First	79	135
Second	81	84
Third	301	104

[183] ✓ 0s

```
sns.countplot(data= clean_df , x="class", hue="survived",
              palette="viridis")
```

```
<Axes: xlabel='class', ylabel='count'>
```





[184]

```
# qu 5- Average passenger class, gender, survived
```

[186] ✓ 0s

```
df.pivot_table(index ="class", columns="sex",
               values="survived", aggfunc="mean",
               fill_value=0)*100
```

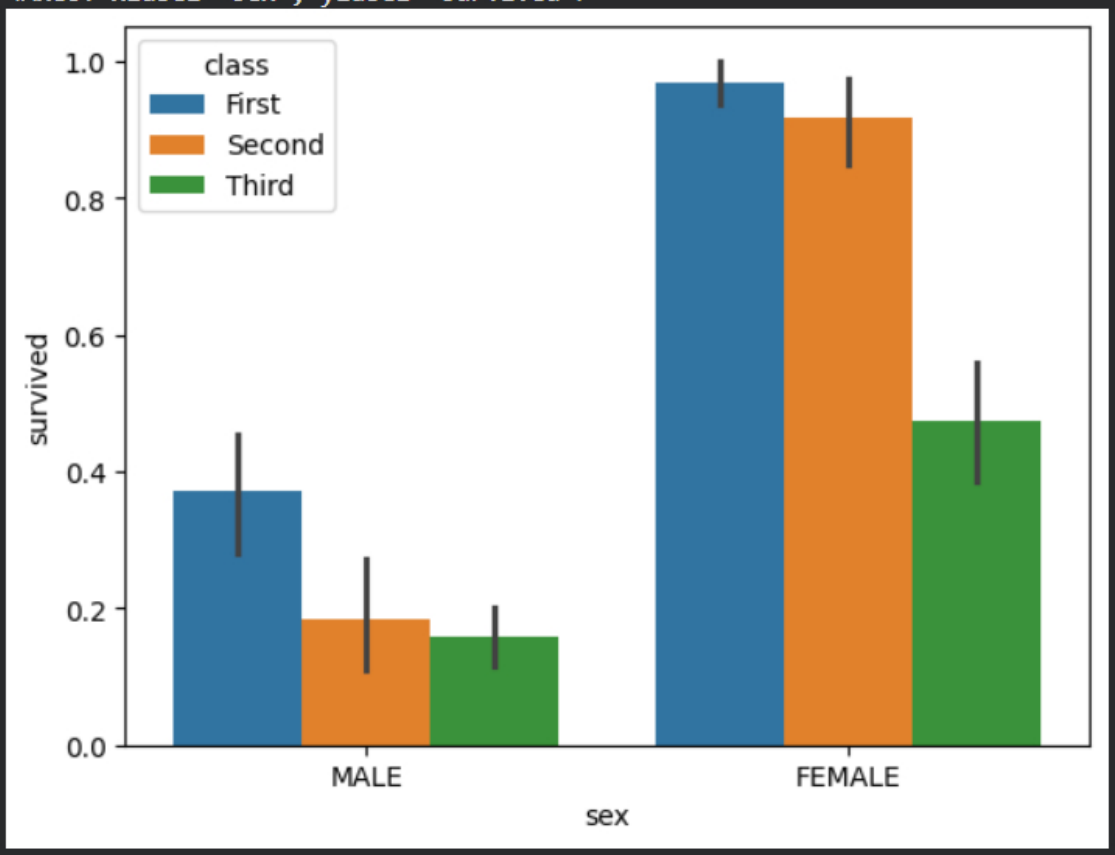
```
/tmp/ipykernel_1455/1082900434.py:1: FutureWarning: The default value of obser
  df.pivot_table(index ="class", columns="sex",
```

class	female	male
First	96.808511	36.885246
Second	92.105263	15.740741
Third	50.000000	13.544669

[188] ✓ 0s

```
sns.barplot(data= clean_df , x="sex", y="survived", hue="class")
```

```
<Axes: xlabel='sex', ylabel='survived'>
```



[189] ✓ 0s

```
# qu 5- age survival rate
```

[191] ✓ 0s

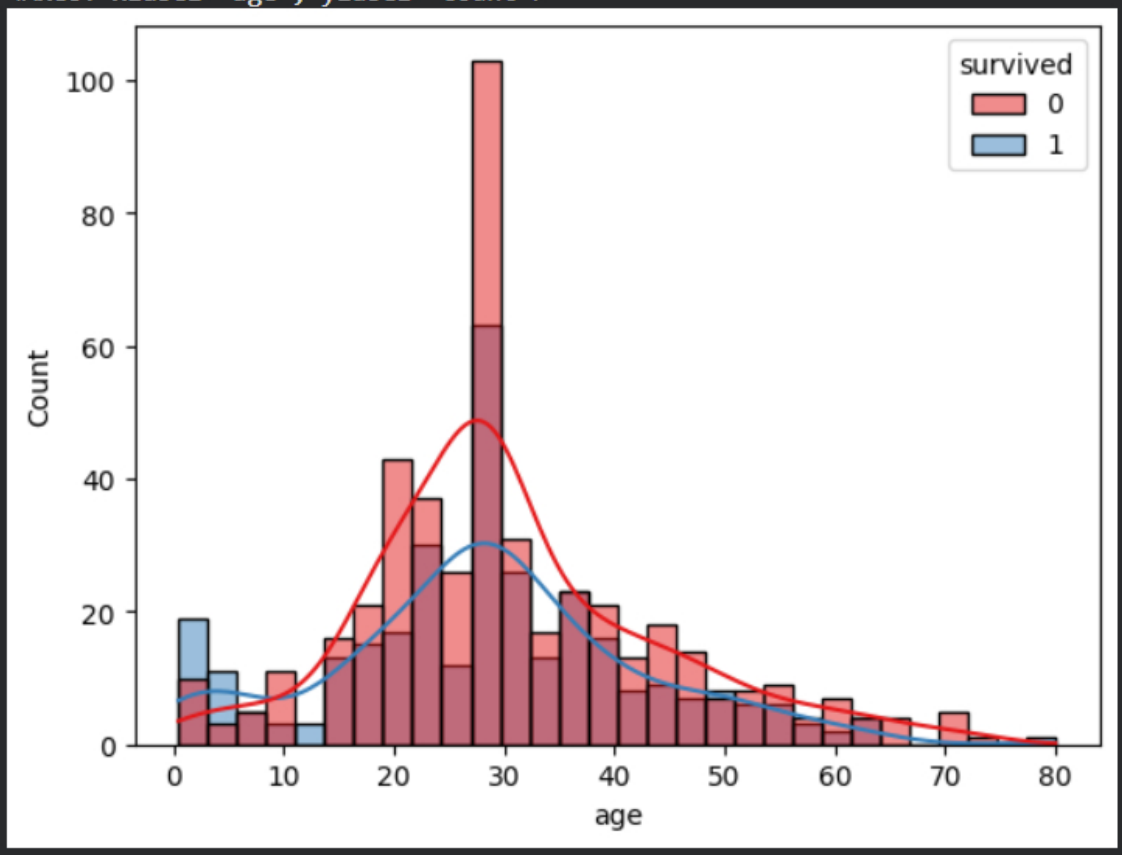
```
clean_df.groupby("survived")["age"].describe()
```

survived	count	mean	std	min	25%	50%	75%	max
0	461.0	30.560195	13.468786	1.00	22.0	28.25	37.0	74.0
1	323.0	28.351920	14.024997	0.42	20.5	28.25	36.0	80.0

[192] ✓ 1s

```
sns.histplot(data=clean_df, x="age", hue="survived",
             bins=30, kde=True, palette="Set1")
```

```
<Axes: xlabel='age', ylabel='Count'>
```



[193] ✓ 0s

```
# qu 7 - age group
```

[194] ✓ 0s

```
clean_df["age_group"].value_counts()
```

age_group	count
adult	435
senior	192

child	68
teen	67
oldage	22

dtype: int64

[196] ✓ 0s

```
clean_df["age_group"].value_counts(normalize=True)*100
```

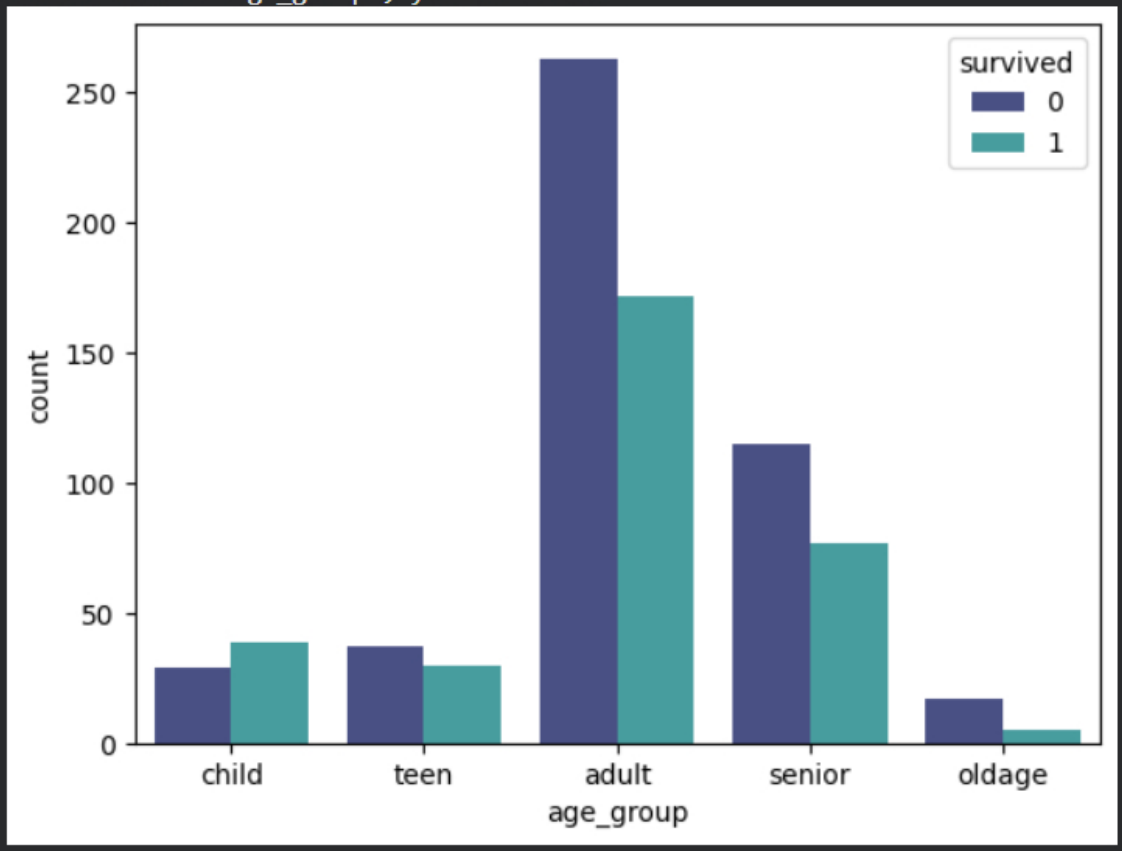
age_group	proportion
adult	55.484694
senior	24.489796
child	8.673469
teen	8.545918
oldage	2.806122

dtype: float64

[198] ✓ 0s

```
sns.countplot(data= clean_df , x="age_group", hue="survived",
              palette="mako")
```

```
<Axes: xlabel='age_group', ylabel='count'>
```



[199] ✓ 0s

```
clean_df.columns
```

```
Index(['survived', 'pclass', 'sex', 'age', 'sibsp', 'parch', 'fare',
       'embarked', 'class', 'who', 'adult_male', 'embark_town', 'alive',
       'alone', 'age_group', 'family_size'],
      dtype='object')
```

[200] ✓ 0s

```
# fare structure
```

[201] ✓ 0s

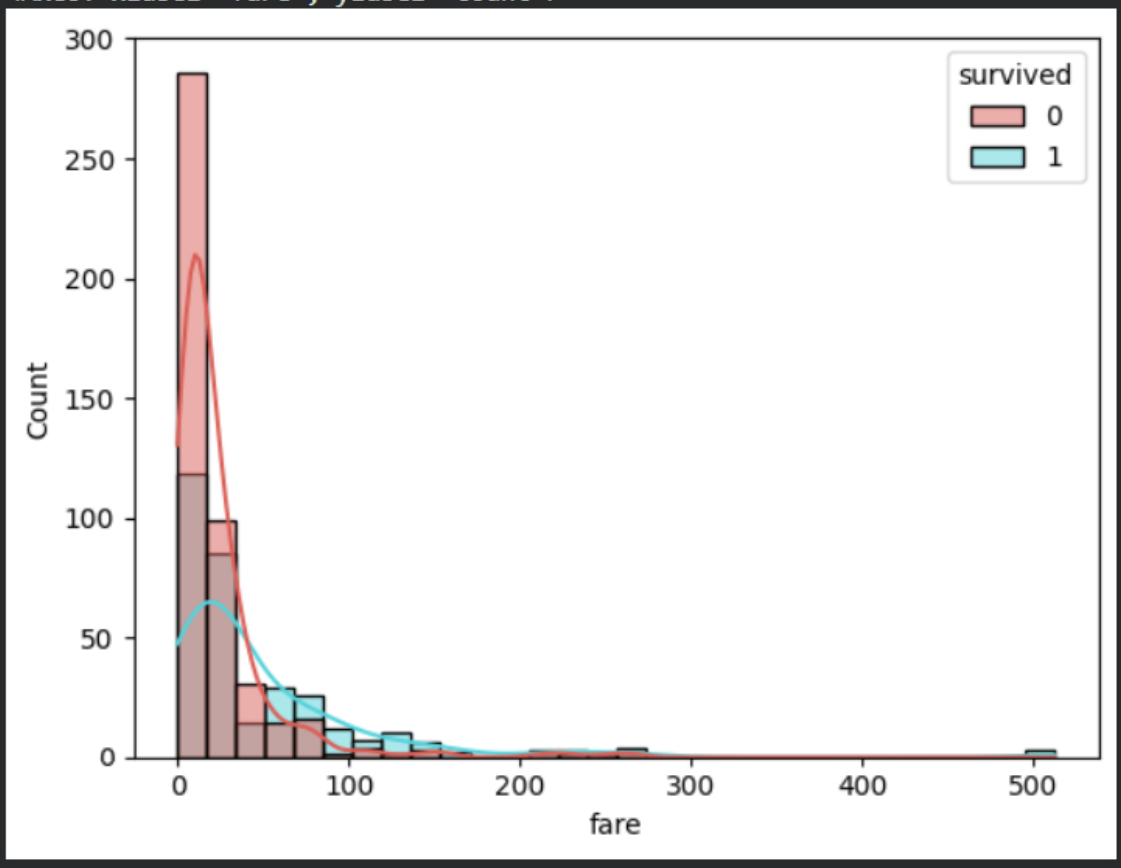
```
df.groupby("alive")["fare"].agg(["count", "mean", "median",
                                 "min", "max"])
```

alive	count	mean	median	min	max
no	549	22.117887	10.5	0.0	263.0000
yes	342	48.395408	26.0	0.0	512.3292

[203] ✓ 1s

```
sns.histplot(data= clean_df, x="fare", hue="survived",
             bins =30,kde=True,  palette="hls")
```

```
<Axes: xlabel='fare', ylabel='Count'>
```



[ ]

```
# 1 - valuecounts
# valuecount(normalize=True) - percentage
# crosstab - cat vs cat
# pivot table - 3 categories
```

```
# group by - category - numerical

# countplot, barplot, histplot
```

Colab paid products - Cancel contracts here



Variables Terminal 11:02 AM Python 3